

Understanding practitioners' strategies to handle test smells: a multi-method study

Railana Santana
Federal University of Bahia
Salvador, Brazil
railana.santana@ufba.br

Daniel David Fernandes
Federal University of Bahia
Salvador, Brazil
daniel.david@ufba.br

Denivan Campos
Federal University of Bahia
Salvador, Brazil
denivan.campos@ufba.br

Larissa Rocha Soares
State University of Feira de Santana
Feira de Santana, Brazil
lrsoares@uefs.br

Rita Maciel
Federal University of Bahia
Salvador, Brazil
rita.suzana@ufba.br

Ivan Machado
Federal University of Bahia
Salvador, Brazil
ivan.machado@ufba.br

ABSTRACT

Test smells are poor design and implementation choices that can affect the test code's understanding and maintainability. Recent studies show the industry is not aware of the test smells concept, while software engineers commonly encounter obstacles to maintain test code. This study investigated test creation and maintenance strategies through developers' perception of eight test smells types. We surveyed 87 software testers and interviewed eight to understand their view on test smells in practice. Our results show that most participants use manual strategies for creating and maintaining test cases. Based on data captured from software testers' perspectives, this study contributes with possible directions and treatments to analyze test smells, seeking to understand how the test affects and potential solutions for test smells.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation.**

KEYWORDS

Survey research, Software testing, Test smells

ACM Reference Format:

Railana Santana, Daniel David Fernandes, Denivan Campos, Larissa Rocha Soares, Rita Maciel, and Ivan Machado. 2021. Understanding practitioners' strategies to handle test smells: a multi-method study. In *Brazilian Symposium on Software Engineering (SBES '21)*, September 27-October 1, 2021, Joinville, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3474624.3474639>

1 INTRODUCTION

Software testing assumes a vital role in the software lifecycle. A well-defined software testing process might improve software quality, reduce software failures, and ensure that the final product meets

the proper requirements [5]. When testing software, we should respect sound design principles so they are easier to understand, maintain and diagnose problems in codifying activities.

Oppositely, test code anti-patterns, known as test smells, indicate poor implementation design, and they can affect the maintainability and understandability of the test code [2]. The recent literature presents many studies on test smells, such as test code development with JUnit framework [3]; the industry's perception on test smells [4]; automated strategies for their identification and removal [8, 11]; and test smells severity [9].

Although several studies on test smells have been conducted recently, aspects related to test code creation and maintenance strategies are still little discussed. Understanding practitioners' preferences and strategies during the test code creation and maintenance process can help academia understand test smells in practice. In this article, we investigate test smells perceptions and strategies on test code adopted by practitioners, such as (i) test code creation and maintenance, (ii) severity, (iii) refactoring practices, and (iv) techniques suitable to improve the test code.

Hence, we conducted a multi-method study composed of two phases: survey and interviews. We applied a survey to 87 software testing practitioners with experience in the JUnit framework. We also performed eight interviews to complement the survey to understand better how test smells can affect the test code and refactor them. We addressed the following research questions:

RQ1: How do practitioners create and maintain test code?

We investigate how practitioners create and maintain test cases (manually or through test generator tools), and the frequency of maintenance of the test code.

RQ2: Are the practitioners aware of test smells?

We investigate practitioners' knowledge of test smells; the frequency that test smells are introduced; how much practitioners estimate the severity of each test smell.

RQ3: How do practitioners deal with test smells?

We investigate the practitioners' opinions on the need to refactor test smells, the frequency, and how they refactor test smells in their projects.

RQ4: What is the practitioners' perception about test quality?

We investigate the practitioners' views on the importance of improving test code quality and the main strategies that could employ in the software development process to improve the test suite's quality.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBES '21, September 27-October 1, 2021, Joinville, Brazil

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-9061-3/21/09...\$15.00

<https://doi.org/10.1145/3474624.3474639>

2 TEST SMELLS

Deursen et al. [2] and Peruma et al. [6] presented a catalog of test smells and refactoring strategies to handle these. For this study, we selected eight of these most frequent test smells in the literature. We next briefly introduce these test smells.

Assertion Roulette (AR). Test methods that present multiple assertions without explanation [2].

Conditional Logic Test (CL). Test methods contain conditional logic statements (e.g., if-else, while, for) [6].

Duplicate Assert (DA). A test method checks the same assertion more than once in the same method but with different values [6].

Exception Handling (EH). This smell occurs when the approval or disapproval of a test method explicitly depends on the production method that throws an exception (try/catch) [6].

Resource Optimism (RO). The test method uses a resource without first checking its state [2].

Sleepy Test (ST). Test method pauses for a certain period by simulating or waiting for an external event (using Thread.sleep()), and then continues execution usually [6].

Unknown Test (UT). The test method that does not contain assertion [6].

Test Run War (TR). Test method allocates resources (e.g., tmp files) also used by other test methods [2].

3 MULTI-METHOD STUDY

In this paper, we applied surveys and interviews so that we could answer our research questions. For the survey, we used objective and quantitative questions. Meanwhile, in the interview, we designed qualitative questions. The RQ₄ was answered only by the interview, and the other RQs were answered using both methods.

3.1 Survey methodology

After identifying the research objectives, we recognized the target audience, composed of professionals with experience in testing with the JUnit framework¹. We selected only Brazilian practitioners. We selected the sample based on three aspects: convenience, social networking platforms (e.g., LinkedIn), and snowballing [12].

The survey comprised 51 closed-ended questions. Among these, 16 had a text field where the participants could justify their answers. We split the questions into five sections: (a) conditional filter to our target audience; (b) demographic information; (c) test creation and maintenance; (d) perspective on test practice patterns (test smells); and (e) knowledge about test smells. We designed the survey in Google Forms. Section (a) had one closed-ended question to select participants with JUnit framework experience. Section (b) had four closed-ended questions corresponds to the participants' profile. Section (c) had four closed-ended questions asks participants how tests are created and maintained, test maintenance frequency, and which test code generation tools they use. In section (d), for each of the 8 test smells investigated, we present a code snippet and a brief description of the practice² without mentioning the test smell name. For each test smell, we asked three closed-ended questions and two questions with an open field. In the last section (e), we

asked two closed-ended questions: whether the participants knew the term "test smell" and which ones they knew.

We invited the participants via e-mail and social networking platforms, e.g., LinkedIn, WhatsApp, Telegram, and Facebook. We distributed the survey from December 3, 2020, to February 25, 2021.

Three of the authors independently extracted the general themes from all responses to each question. Using these themes, the authors held discussion sessions to reach an agreement.

3.2 Interview methodology

We conducted a pilot interview to validate our questions and analyze their length. We interviewed one researcher with a degree in Computer Science and experience in software development and the JUnit framework for this pilot.

After obtaining survey responses, we sent 31 emails scheduling interviews with participants interested in collaborating with the second phase of the study, while only 8 returned the email confirming the interviews. All interviews were conducted via Google Meet. During interviews, participants consented to record and agreed to the disclosure of the results under anonymity. Interviews ranged from 40 to 60 minutes.

We listened to the recordings and transcribed the entire conversation. To transcribe the audio, we had the support of the Transcriber-Bot³ tool for Telegram. Then, we investigated the participants' responses for each question and created the codings to classify and group responses according to each researcher. In case of disagreements among codings, we analyzed and decided which coding best suited to those answers.

4 RESULTS

After analyzing and removing invalid responses from the survey, we obtained 87 valid responses. The participants had different levels of education: 2 (2.30%) High school completed, 19 (21.84%) participants were undergraduate students, 30 (34.48%) were B.Sc., 18 (20.69%) had MBA, 5 (5.75%) were M.Sc. students, 7 (8.05%) M.Sc., and 6 (6.90%) were Ph.D. students. All survey participants had experience in the industry, and more than half had more than five years of experience. Regarding the JUnit framework, 19.54% had up to 1 year of experience, 56.32% had up to 5 years, 12.64% had up to 10 years, and 11.49% had more than ten years of experience.

After applying the survey, we interviewed a subgroup of survey participants with eight professionals: 6 participants had a Bachelor's degree, 1 had an MBA in technology, and 1 was a Master's student. Most of them had experience with development for more than five years. Regarding their experience with JUnit, seven had up to five years of experience and one participant up to one year of experience.

4.1 Test code creation and maintenance (RQ1)

In the survey, we were interested in understanding whether practitioners use test code generation tools for test case creation/maintenance and how often tests are maintained.

The survey results showed that for creating the tests: 60.91% of the participants manually create test cases, 32.18% use manual & automated strategies, and 5.74% adopt exclusively automated strategies. For maintaining the test code, 72.41% perform manual

¹<https://junit.org>

²<https://zenodo.org/record/5115285#.YPYzSehKiUk>

³<https://github.com/charslab/TranscriberBot>

maintenance, 21.83% use both manual & automated, and 1.14% only use automated strategies. We also obtained inconclusive answers in the *Others* field (1.14% for creation and 4.59% for maintenance). Thus, we can observe a preference for creating/maintaining test code manually.

To investigate the maintenance frequency of the tests, we used the 5-point Likert scale. 14.9% of the participants reported they perform maintenance "All the time ($\cong 100\%$)"; 26.4% informed "Often ($\cong 75\%$)"; 37.9% "Sometimes ($\cong 50\%$)"; 20.7% reported they perform maintenance "Rarely ($\cong 25\%$)"; nobody reported "Never ($\cong 0\%$)". All participants claimed they perform maintenance on test code at least rarely. This finding reinforces the importance of good practices during the test code implementation since a test code may not be maintained by the same developer who created it.

4.2 Awareness about test smells (RQ2)

In the survey, we investigated participants' knowledge of the term test smells, knowledge of their types, the frequency with which test smells are entered, and the severity of the test smell. To better understand the choice of severity, we asked participants to explain how test smells affect the test code during the interview.

In the form, we asked the participants whether they had already heard about test smells and what types they know. We observed that most of the participants (68.96%) had already heard about test smells, different from the Deursen et al. [2] study, indicating that the industry has learned more about test smells in the last five years.

The best-known test smells types are EH (37.93%), CL (36.78%), ST (31.03%), and AR (20.69%). We note that participants also know other test smells that we did not explore for this study (e.g., Duplicate Assert and Empty Test).

We asked how often the participants write test codes similar to those test smells. They could choose one of the following answers: Never ($\cong 0\%$), Rarely ($\cong 25\%$), Sometimes ($\cong 50\%$), Often ($\cong 75\%$), and All the time ($\cong 100\%$). Table 1 presents the answers. For instance, AR is the most inserted "All the time", according to 13.79% of participants, and this finding converges with other studies [6, 7].

Table 1: Frequency that test smells are inserted (%)

Frequency	AR	CL	DA	EH	RO	ST	UT	TR
Never	29.89	56.32	34.48	44.83	34.48	54.02	79.31	64.37
Rarely	27.59	31.03	26.44	19.54	29.89	25.29	12.64	14.94
Sometimes	13.79	11.49	20.69	24.14	18.39	9.20	6.90	10.34
Often	14.94	1.15	14.94	9.20	13.79	11.49	1.15	8.05
All the time	13.79	0.00	3.45	2.30	3.45	0.00	0.00	2.30

We used the 5-point Likert scale to classify severity as "None", "Few severe", "Moderately severe", "Severe", and "Very severe". Table 2 shows the level of severity for each test smell. For instance, we observed that 44.83% of participants considered the test smell AR as a few severe for the test code, and the CL was considered very severe by 37.93%.

In the interview, we were also interested in understanding how the test is affected by each type of practice. We present the participants' statements for each test smell:

Table 2: Level of severity of test smells (%)

Severity	AR	CL	DA	EH	RO	ST	UT	TR
None	6.90	2.30	17.24	14.94	9.20	11.49	13.79	10.34
Few severe	44.83	9.20	34.48	41.38	36.78	19.54	20.69	22.99
Moderately	25.29	25.29	32.18	31.03	20.69	31.03	13.79	12.64
Severe	10.34	25.29	9.20	5.75	22.99	14.94	16.09	22.99
Very severe	12.64	37.93	6.90	6.90	10.34	22.99	35.63	31.03

AR. It makes it hard to read, understand the expected result, and reuse the code. They informed that it is preferable to have smaller methods with fewer assertions than a single method with several ones.

CL. Using conditional structures in the test code affects readability, increases the method's complexity, and makes it difficult to understand the method objective. The method's behavior can be changed using these structures. The test can pass without actually doing the correct validation, generating tests with non-deterministic behavior. Participants also reported that the single responsibility principle is not followed when using this practice.

DA. Participants explained that having assertions with equal parameters in the same method affects the test objective understanding and readability and hurts the single responsibility principle.

EH. Participants believe that using *try/catch* in the test method's body makes it difficult to understand the test's purpose, affecting test readability. In addition, a test with *try/catch* structures becomes more verbose and consequently increases the complexity. Participants also reported that this practice could have an impact similar to using conditionals in a test.

RO. They claimed that using external resources without first checking their status could be harmful as it impacts the overall quality. It can be challenging to predict resources' states, as it can present a non-deterministic behavior (Flaky Test [14]), being hard to identify the root problem. Moreover, it could cause an exception (e.g. *Null Pointer Exception*).

ST. They believe that it affects the test because it impacts the test execution time and performance.

UT. According to them, a test method with no assertions does not check anything; the objective of the method is implicit. Therefore, it does not make sense to keep this method. This practice was considered a developer error and violated the principle of validating an automated test.

TR. Using this type of practice creates a dependency on the order in which tests are performed, violating first principles and allowing cascading error to occur; affects maintenance during the evolution of the system; increases the coupling among tests; creates resource competition among test methods; and impacts the maintenance of the test and production code. It can also make it difficult to identify the reasons why the test failed.

4.3 How test smells are handled (RQ3)

We asked the survey respondents whether they refactor test smells and how often they do it. In addition, in the interview, we asked how they commonly refactor the test code.

Table 3 shows the results of the participants' opinions about the refactoring of each test smell type. In general, there was more agreement than disagreement for refactorings. We observed unanimity regarding the refactoring of UT and CL. 89.66% and 87.36% of participants, respectively, agreed with their refactorings. Meanwhile, the refactorings that most presented disagreements were: EH test smells (32.18% of participants) and DA (27.59% of participants).

Table 3: Opinion on the need to refactor test smells (%)

Response	AR	CL	DA	EH	RO	ST	UT	TR
Refactor	72.41	87.36	63.22	62.07	65.52	77.01	89.66	77.01
Don't refactor	19.54	9.20	27.59	32.18	25.29	17.24	5.75	14.94
Don't know	0.00	0.00	0.00	0.00	1.15	0.00	1.15	2.30
Inconclusive	8.05	3.45	9.20	5.75	8.08	5.75	3.45	5.75

Table 4 shows the frequency in which each test smell is refactored. 33.33% of participants reported that they *never* ($\cong 0\%$) refactor UT and TR test smells. On the other hand, for 21.84%, the UT is refactored *all the time* ($\cong 100\%$). We found that the most refactored ones are AR and CL test smells.

Table 4: Frequency of test smells refactoring (%)

Refactoring	AR	CL	DA	EH	RO	ST	UT	TR
Never	11.49	20.69	25.29	24.14	24.14	24.14	33.33	33.33
Rarely	24.14	21.84	28.74	42.53	36.78	28.74	29.89	21.18
Sometimes	31.03	25.29	29.89	17.24	18.39	20.69	6.90	17.24
Often	24.14	17.24	12.64	8.05	12.64	13.79	8.05	4.60
All the time	9.20	14.94	3.45	8.05	8.05	12.64	21.84	12.64

We showed the test code snippets during the interview and asked the need to refactor them. All participants agreed that the test cases should be refactored, except for AR, CL, and EH (only 1 participant did not find it necessary). When the participant agreed that they needed to refactor, we asked them which strategy they would use. As the refactoring of a test smell might depend on the context, the participants suggested more than one refactoring strategy.

AR. Five participants suggested splitting the method into others to avoid multiple assertions within the same test method, and three participants suggested including the explanation parameter.

CL. Four participants reported that they could do the refactoring by splitting the method into more methods to reach the conditional structures. Two participants recommended refactoring with a content abstraction of the conditional structure in an auxiliary method.

DA. Five participants suggested refactoring it by dividing the original method into more test methods for each new value that the variable assumes.

EH. Three participants suggested dividing the test method into more methods, where each method individually validates success and exception. Two participants reported that it was necessary to analyze the production method to make any refactoring decision. One participant would remove the *try/catch* and evaluate the need to insert *throws*. One participant informed that the refactoring

could be done with *Assumptions*⁴ to ignore the test method under certain conditions. Furthermore, other participant recommended using *assertThrows()* and *assertException()* methods to validate the expected behavior.

RO. Six participants informed that before using the resource, they should do a check. Four participants suggested using abstractions for the resource (e.g., mock). Three participants suggested creating the resource using the *setup* method. One participant claimed he would use *jUnit* resources to handle temporary files. Furthermore, one participant suggested splitting into more tests to verify a possible exception launch.

ST. Four participants said they would use an intelligent waiting library (e.g., Awaitility⁵). Three participants would make the request asynchronous (e.g., mock). One participant would separate it in a method with a test step in a more indicative place. Besides, other participants suggested ordering the tests' execution, adding tests containing thread. Sleep command at the end of the test suite.

UT. All participants said they would refactor a test method without assertions by including an assertion in the test method. They also reported removing this test method depending on its purpose (or lack of purpose).

TR. Although all participants agreed that this practice could impact the test code quality, only five agreed they should refactor it. Three participants suggested ordering the execution of the test methods; three suggested using *jUnit*'s annotations (*@before* and *@after*) and declaring the feature in the setup. One participant mentioned using the *jUnit TemporaryFolder* rule to create specific files for each test method. One participant would refactor it by splitting the class into other classes to contemplate each functionality in a different class. Furthermore, one participant suggested refactoring by grouping the methods into a single test method.

4.4 Quality improvement strategies (RQ4)

All the participants reported that it is necessary to evaluate the test code quality. Each one argued positive points in evaluating test code quality. Among the explanations, the test code should maintain the same quality standard as the production code. Two of them commented that a good test code directly influences good production code. Four of them pointed out that test code should follow good practices because other developers would use it and maintain it.

According to the suitable strategies to improve test code quality, three participants mentioned that the company's culture significantly influences test code quality. Two participants said that training is essential to disseminate good practices in the workplace. Two participants talked about the practice of code review, in which one or more developers check code quality written by someone else. Two participants reported the need for tools to analyze the code from the perspective of readability and maintenance. One participant each mentioned the following strategies: refactoring the test code frequently; improve the test pyramid (unit tests at the bottom of the pyramid, integration test in the middle of the pyramid, and graphical interface test at the top), enhance the test code quality as well as the production code, and define a standard to be followed within the company.

⁴<https://junit.org/junit5/docs/5.0.0/api/org/junit/jupiter/api/Assumptions.html>

⁵<https://github.com/awaitility/awaitility>

Furthermore, two participants reported they should write tests with low complexity and coupling. For two participants, tests should be easy to read. One participant mentioned other strategies related to the test structure: write clean test code, optimize the test execution time, and improve test parallelization and component reuse.

The interview results show that company culture and training can be great allies in the quest for test code quality. When a company incorporates best practice principles, the pursuit of test quality becomes more natural. We also realize that some practical concepts, such as the single responsibility principle and clean code test, can avoid more than one test smell.

5 RELATED WORK

To understanding testing activities practices, several studies have been carried out. According to Yamashita and Moonen [13], the authors conducted an exploratory and descriptive survey to understand the level of knowledge and the importance of code smells for developers. In the study by De Bleser et al. [1], the authors assessed the perception and the ability to identify test smells by developers on SCALA projects. The study's objective is to evaluate the knowledge of test smells and the perception of their severity by the developers of SCALA projects. Spadini et al. [9] examined the severity classification for four test smells and investigated its perceived impact on the test suite ability to be maintained by developers. Junior et al. [4] surveyed the test practitioners' perception of test smells. This survey aimed to find out if test professionals insert test smell unintentionally. According to Tufano et al. [10], the authors conducted an empirical study on test smells in which they evaluated the developer's perceptions and how they handle code affected by some test smells.

Our work focuses on source code, specifically JUnit, to understand how software testers handle test smells regarding their treatment and severity. Our proposal aims to analyze developers' perception about 8 test smells, in which 2 of these were analyzed by Spadini et al. [9]. Like in the study of Junior et al. [4], we did not mention the term test smell to avoid inducing participants. However, we used code snippets to exemplify the investigated practices (test smells). In this study, we investigated similar topics that have been explored individually in the related studies. However, we sought to analyze several topics about test code and test smells in a single study. Thus, we can compile developers' strategies through a multi-method study to understand how they handle test smells according to their severity level.

6 CONCLUSION

In this study, we investigated the main strategies for creating and maintaining code and testing and the effects of test smell on the maintenance and test code quality from the practitioners' perspective with experience in software testing. We gathered data from 87 practitioners and interviewed eight of them who hold experience in software testing.

The results show that 72.41% of the practitioners have a preference to create and maintain test code manually. 63.22% of the practitioners know some test smells. Of the eight types of test smells investigated, 3 test smells (AR, DA, and RO) are frequently inserted into the test cases. According to the severity analysis, the

participants considered CL, UT, and TR test smells very severe. Although the test smells have different severity levels, 83.3% of the participants agree with refactoring the 8 test smells investigated.

Future work would be interesting to investigate the testers' perspective on other programming languages and testing frameworks. It is interesting to examine whether the test smells behavior is applicable in different domains. In addition, analyze multiple scenarios of the same test smell type to verify refactoring proposals in other cases.

ACKNOWLEDGMENTS

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001 and FAPESB grants BOL0599/2019 and JCB0060/2016.

REFERENCES

- [1] Jonas De Bleser, Dario Di Nucci, and Coen De Roover. 2019. Assessing diffusion and perception of test smells in scala projects. In *16th International Conference on Mining Software Repositories (MSR)*. IEEE, Montreal, QC, Canada, 457–467.
- [2] Arie Van Deursen, Leon Moonen, Alex Van Den Bergh, and Gerard Kok. 2001. Refactoring test code. In *2nd international conference on extreme programming and flexible processes in software engineering (XP2001)*. CWI (Centre for Mathematics and Computer Science), NLD, 92–95.
- [3] Vahid Garousi and Barış Küçük. 2018. Smells in software test code: A survey of knowledge in industry and academia. *Journal of systems and software* 138 (2018), 52–81.
- [4] Nildo Silva Junior, Larissa Rocha Soares, Luana Almeida Martins, and Ivan Machado. 2020. A survey on test practitioners' awareness of test smells. *CoRR abs/2003.05613* (2020).
- [5] Mohd Ehmer Khan and Farneena Khan. 2014. Importance of software testing in software development life cycle. *International Journal of Computer Science Issues (IJCSI)* 11 (2014), 120.
- [6] Anthony Peruma, Khalid Almalki, Christian D Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2019. On the distribution of test smells in open source android applications: An exploratory study. In *29th Annual International Conference on Computer Science and Software Engineering*. IBM Corp., USA, 193–202.
- [7] Anthony Peruma, Khalid Almalki, Christian D Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2020. Tsdetect: An open source test smells detection tool. In *Proceedings of the 28th Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, NY, USA, 1650–1654.
- [8] Railana Santana, Luana Martins, Larissa Rocha, Tássio Virginio, Adriana Cruz, Heitor Costa, and Ivan Machado. 2020. RAIDE: a tool for Assertion Roulette and Duplicate Assert identification and refactoring. In *Proceedings of the XXXIV Brazilian Symposium on Software Engineering - Tools Track*. ACM, NY, USA, 374–379.
- [9] Davide Spadini, Martin Schvarcbacher, Ana-Maria Oprea, Magiel Bruntink, and Alberto Bacchelli. 2020. Investigating severity thresholds for test smells. In *Proceedings of the 17th International Conference on Mining Software Repositories*. ACM, Seoul, Republic of Korea.
- [10] Michele Tufano, Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Andrea De Lucia, and Denys Poshyvanyk. 2016. An empirical investigation into the nature of test smells. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Singapore.
- [11] Tássio Virginio, Luana Martins, Larissa Rocha, Railana Santana, Adriana Cruz, Heitor Costa, and Ivan Machado. 2020. JNose: Java Test Smell Detector. In *Proceedings of the XXXIV Brazilian Symposium on Software Engineering - Tools Track*. ACM, NY, USA, 6 pages.
- [12] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. ACM, NY, USA, 10 pages.
- [13] Aiko Yamashita and Leon Moonen. 2013. Do developers care about code smells? An exploratory survey. In *20th working conference on reverse engineering (WCORE)*. IEEE, Koblenz, Germany.
- [14] Behrouz Zolfaghari, Reza Parizi, Gautam Srivastava, and Yoseph Hailemariam. 2020. Root causing, detecting, and fixing flaky tests: State of the art and future roadmap. *Software Prac. Experience* 51 (2020).